

Pokemo

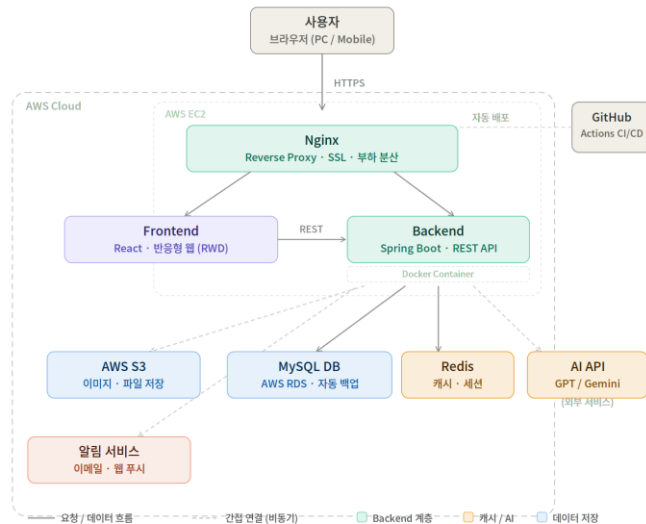
설계 문서

Design Document — Architecture, Class & Sequence Diagrams

프로젝트명	Pokemo — AI 기반 학습 관리 플랫폼
의뢰사	EduNext Solutions
작성팀	개발팀 (26SS Software Engineering)
문서 버전	v3.4
작성일	2026-05-24
산출물 단계	1단계 (12주차까지) — 설계 문서
포함 내용	시스템 아키텍처 개요 · 클래스 다이어그램 · 시퀀스 다이어그램 4종

1. 시스템 아키텍처 개요

Pokemo는 React 기반 Frontend, Spring Boot 기반 Backend, MySQL DB로 구성된 3-Tier 웹 아키텍처다. 모든 컴포넌트는 AWS 클라우드 위에서 Docker 컨테이너로 운영되며, Nginx가 리버스 프록시 역할을 담당한다. GPT/Gemini API를 통해 AI 기능을 제공하고, GitHub Actions로 CI/CD를 자동화한다.



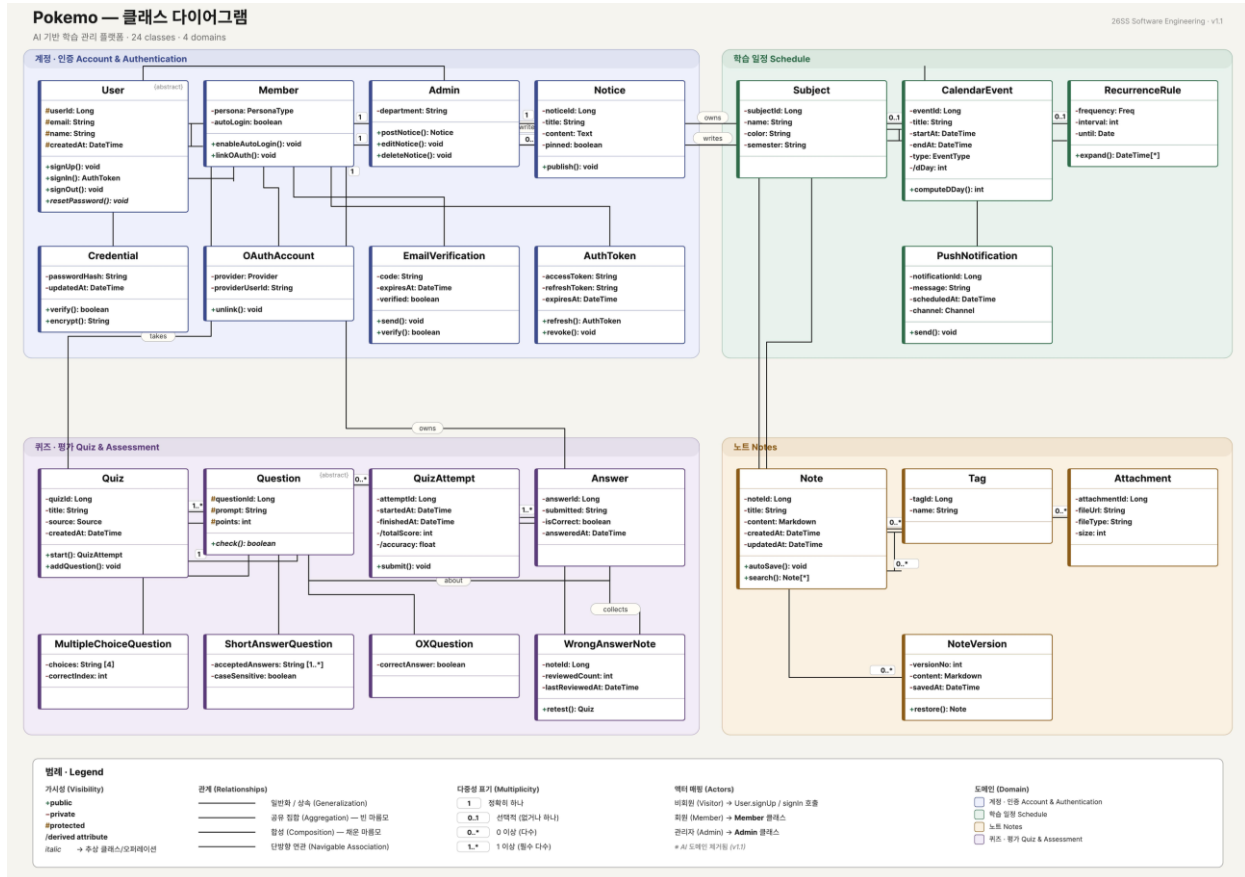
[그림 1-1] Pokemo 시스템 아키텍처 구성도

컴포넌트 역할

레이어	기술 스택	역할
Proxy	Nginx	리버스 프록시, SSL/TLS 종단, 부하 분산, 정적 파일 서빙
Frontend	React (SPA)	사용자 인터페이스, 반응형 웹(RWD), REST API 호출
Backend	Spring Boot	비즈니스 로직, RESTful API, JWT 인증, AI API 서버 호출
AI 서비스	GPT / Gemini API	퀴즈 자동 생성, 학습 요약, 취약 개념 분석 (별도 계층 분리)
DB	MySQL (RDS)	핵심 데이터 영구 저장, RDS 자동 백업
캐시	Redis	반복 조회 캐싱, 세션 관리
파일	AWS S3	이미지·파일 저장
알림	이메일·웹 푸시	학습 일정 임박 알림 발송
인프라	AWS EC2 + Docker	컨테이너 기반 운영, 배포 환경 일관성 확보
CI/CD	GitHub Actions	자동 빌드·테스트·배포 파이프라인

2. 클래스 다이어그램

Pokemo 서비스의 도메인 모델을 UML 클래스 다이어그램으로 나타낸 것이다. 총 24개 클래스, 4개 도메인(계정:인증, 학습 일정, 노트, 퀴즈·평가)으로 구성된다.



[그림 2-1] Pokemo 클래스 다이어그램

도메인별 주요 클래스

도메인	주요 클래스	설명
계정·인증	User, Member, Admin, Credential, AuthToken	추상 클래스 User를 Member-Admin이 상속. JWT 기반 인증, 소셜 로그인(OAuthAccount), 이메일 인증 포함
학습 일정	CalendarEvent, Subject, RecurrenceRule, PushNotification	과목(Subject)에 귀속된 일정. 반복 일정 및 웹 푸시 알림 포함
노트	Note, Tag, Attachment, NoteVersion	Markdown 기반 노트. 태그·첨부파일 연관, 버전 이력 포함
퀴즈·평가	Quiz, Question, Answer, QuizAttempt, WrongAnswerNote, 3종 Question 하위	추상 클래스 Question을 객관식·단답형·OX가 상속. 퀴즈 시도·답변·오답노트(WrongAnswerNote) 결과 기록. QuizAttempt의 startedAt-finishedAt으로 학습 시간 집계 가능. 전반적인 학습 세션 기록은 추후 StudySession 클래스 확장 예정 (F-31).

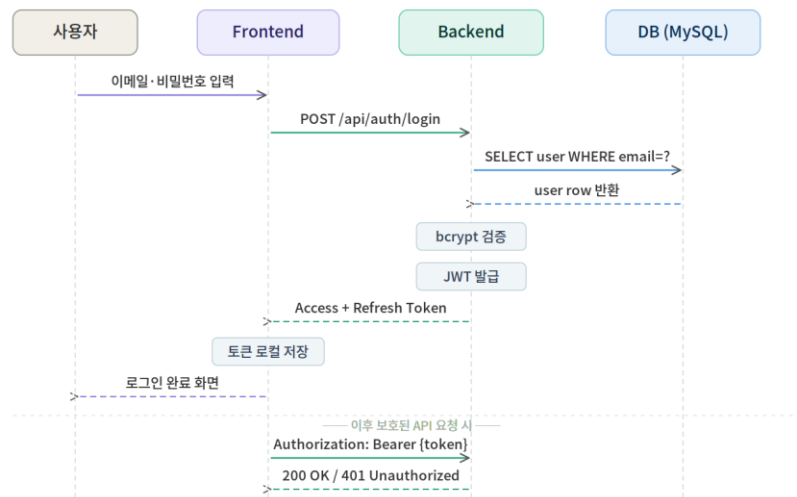
3. 시퀀스 다이어그램

주요 기능 흐름 4종을 UML 시퀀스 다이어그램으로 표현했다. 실선은 요청, 점선은 응답, 회색 박스는 내부 처리를 나타낸다.

3.1 로그인 / JWT 인증

이메일·비밀번호 입력 → bcrypt 검증 → JWT Access/Refresh Token 발급 → 이후 API 요청 시 토큰으로 사용자 검증.

참여자	사용자, Frontend, Backend, DB
트리거	로그인 버튼 클릭
예외	비밀번호 불일치 → 401 / 이메일 없음 → 404
관련 요구사항	F-03 로그인, F-04 자동 로그인, NF-08 JWT 보안



[그림 3-1] 로그인 / JWT 인증

흐름 상세 설명

로그인 흐름은 크게 최초 인증과 이후 요청 검증 두 단계로 나뉜다. 최초 인증에서는 사용자가 입력한 비밀번호를 bcrypt로 검증하고 JWT를 발급한다. 이후 요청부터는 DB 조회 없이 토큰만으로 사용자를 식별하므로 서버 부하가 줄어든다.

핵심 기술 포인트

- bcrypt 단방향 해시 (NF-07): 비밀번호를 원문 그대로 저장하지 않고 해시값으로 저장한다. DB가 유출되더라도 원본 비밀번호 노출을 방지한다.
- JWT 이중 토큰 구조 (NF-08): Access Token(단기·15~30분)과 Refresh Token(장기·7~14일)을 분리 발급한다. Access Token 만료 시 Refresh Token으로 재발급하여 자동 로그인(F-04)을 지원한다.
- Role 기반 인가 (F-07): Access Token 페이로드에 사용자의 Role(USER/ADMIN) 정보를 포함하여 발급한다. 이후 보호된 API 요청 시 서버는 DB 조회 없이 토큰의 Role 클레임만으로 권한 인가(Authorization)를 처리할 수 있다.
- Stateless 인증: 서버가 세션 상태를 저장하지 않아 수평 확장(scale-out)에 유리하다. 초기 3,000명 이상으로 성

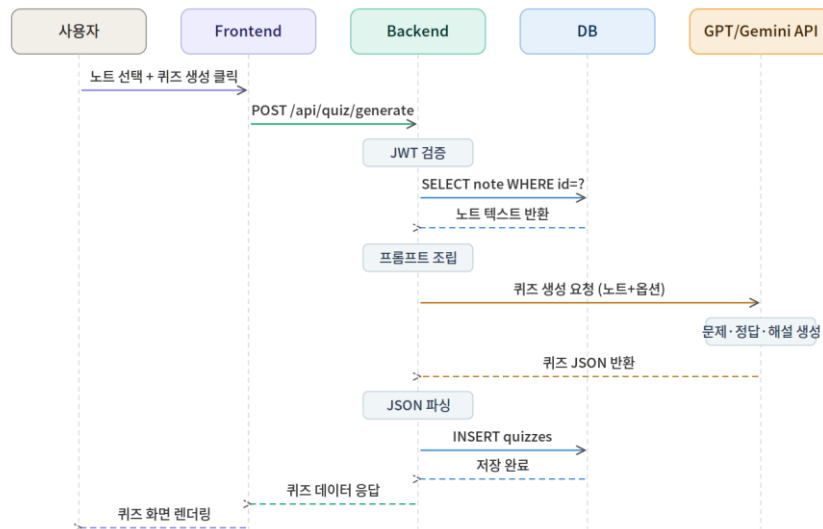
장할 때 서버 추가만으로 대응 가능하다.

- 보안 강화 (NF-06): HTTPS 적용으로 토큰 탈취를 방지하며, 401 응답 시 Frontend에서 자동으로 토큰 갱신을 재시도한다.

3.2 AI 퀴즈 자동 생성

노트 선택 → JWT 검증 → 노트 조회 → 프롬프트 조립 → GPT/Gemini API 호출 → 퀴즈 JSON 파싱 → DB 저장 → 화면 렌더링.

참여자	사용자, Frontend, Backend, DB, GPT/Gemini API
트리거	'퀴즈 생성' 버튼 클릭
예외	AI 타임아웃 → 503 / 노트 없음 → 404
관련 요구사항	F-25 AI 문제 자동 생성, F-23 점수 기록



[그림 3-2] AI 퀴즈 자동 생성

흐름 상세 설명

노트 기반 AI 퀴즈 생성은 Pokemo의 핵심 차별 기능이다. Backend가 GPT/Gemini API를 직접 호출하는 서버 사이드 방식을 채택하여 API 키가 클라이언트에 노출되지 않는다. 생성된 퀴즈는 JSON으로 파싱된 뒤 즉시 DB에 저장되어 이후 오답노트·정답률 분석 등 연계 기능에 활용된다.

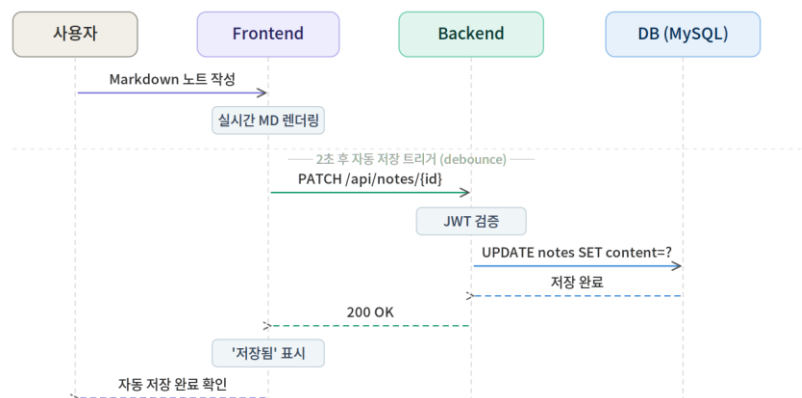
핵심 기술 포인트

- 서버 사이드 AI 호출: Frontend가 GPT/Gemini API를 직접 호출하지 않고 Backend를 경유한다. API 키 노출 방지와 함께 프롬프트 품질 관리, 호출 횟수 제한 등 제어가 용이하다.
- AI 서비스 계층 분리 (NF-21): AI 호출 로직을 별도 서비스 계층으로 분리하여 모델 교체 시 해당 계층만 수정하면 되도록 설계했다.
- 프롬프트 조립 전략: 노트 원문에 문제 유형(객관식·단답형·OX)과 난이도 옵션을 함께 전달한다. 응답 형식을 JSON으로 강제하여 파싱 실패율을 최소화한다.
- 예외 처리 흐름: AI API 타임아웃(503) 또는 JSON 파싱 실패 시 사용자에게 오류 메시지를 반환하고 DB INSERT는 수행하지 않아 불완전 데이터 저장을 방지한다.

3.3 노트 작성 + 자동 저장

Markdown 입력 → 실시간 렌더링 → 2초 debounce → PATCH 요청 → JWT 검증 → DB UPDATE → '저장됨' 표시.

참여자	사용자, Frontend, Backend, DB
트리거	입력 후 2초 경과 (debounce)
예외	네트워크 오류 시 재시도 / JWT 만료 시 자동 갱신
관련 요구사항	F-13 Markdown 편집, F-18 자동 저장



[그림 3-3] 노트 작성 + 자동 저장

흐름 상세 설명

노트 자동 저장은 사용자가 저장 버튼을 누르지 않아도 입력이 멈춘 직후 자동으로 DB에 반영되는 기능이다. 매 키 입력마다 API를 호출하면 서버 부하가 급증하므로 debounce 기법을 적용하여 마지막 입력 후 2초가 지난 시점에만 요청을 전송한다.

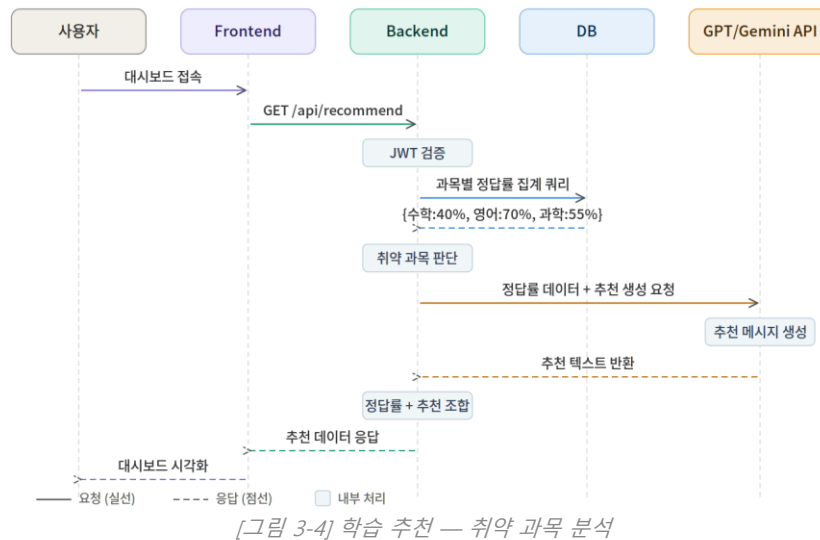
핵심 기술 포인트

- Debounce 패턴: 사용자가 타이핑하는 동안에는 타이머를 계속 리셋하고, 입력이 멈춘 뒤 2초 후에만 PATCH 요청을 전송한다. 10초간 연속 타이핑해도 API 호출은 단 1회만 발생하여 서버 부하를 최소화한다 (NF-01).
- PATCH 방식 사용: 노트 전체를 재전송하는 PUT 대신 변경된 content 필드만 전송하는 PATCH를 사용한다. 대용량 노트에서도 네트워크 비용을 최소화할 수 있다.
- 실시간 Markdown 렌더링: 저장 요청과 별개로 Frontend에서 입력 즉시 Markdown을 렌더링하여 실시간 미리보기를 제공한다. 저장 상태와 렌더링 상태를 분리 관리한다.
- 저장 상태 표시: 저장됨 / 저장 중... / 저장 실패 세 가지 상태를 UI에 표시하여 사용자가 데이터 유실 없이 작업 중임을 인지할 수 있게 한다 (F-18).

3.4 학습 추천 — 취약 과목 분석

대시보드 접속 → 과목별 정답률 집계 → 취약 과목 판단 → GPT/Gemini API로 추천 생성 → 정답률 + 추천 조합 → 시각화.

참여자	사용자, Frontend, Backend, DB, GPT/Gemini API
트리거	대시보드 접속
예외	AI 오류 시 정답률만 표시 / 데이터 없을 시 안내 메시지
관련 요구사항	F-29 취약 과목 추천, F-32 과목별 성취도, F-34 학습 대시보드



흐름 상세 설명

학습 추천 흐름은 규칙 기반 추천과 AI 기반 자연어 추천을 결합한 구조다. DB에서 집계한 정답률 데이터(과목별 정답률)로 취약 과목을 먼저 판단한 뒤, 이 데이터를 GPT/Gemini API에 전달하여 사용자가 이해하기 쉬운 자연어 추천 메시지를 생성한다. AI API 오류가 발생해도 정답률 데이터는 항상 표시되도록 분리 설계했다.

핵심 기술 포인트

- 2단계 추천 구조: 1단계로 DB 집계 쿼리로 정답률을 계산하고, 2단계로 AI API를 호출해 추천 메시지를 생성한다. AI 오류 시에도 정답률 데이터는 정상 표시된다 (Graceful Degradation 패턴).
- Redis 캐시 활용 (NF-04): 대시보드 접속 시 매번 DB를 집계하지 않고 Redis에 캐싱된 정답률 통계를 우선 조회한다. 캐시 미스(Cache Miss) 시에만 DB를 집계하여 응답 속도를 개선한다. QuizAttempt.submit() 시점에 해당 과목의 캐시를 비동기로 갱신(Cache Invalidation)하여 데이터 정합성을 유지한다.
- 진척도 계산 로직: 과목별 진척도(%)는 해당 과목 누적 정답 수 ÷ 누적 문제 수 × 100으로 계산한다. 최근 퀴즈 일수록 가중치를 높여 현재 실력을 더 정확하게 반영한다.
- D-Day 연동 추천 (F-30): 시험 임박 과목 추천은 캘린더 일정(CalendarEvent)의 D-Day 값을 함께 조회하여 정답률이 낮고 시험이 임박한 과목을 최우선으로 추천한다.
- AI 프롬프트 설계: 정답률 JSON 데이터와 함께 각 과목에 대한 학습 추천 메시지를 한국어로 작성하도록 프롬프트를 전달한다. 응답 길이를 제한하여 UI 카드에 맞는 분량의 텍스트만 반환받는다.

※ 본 문서는 *Pokemo* 요구사항 명세서를 기반으로 개발팀이 작성한 설계 문서입니다.